

AppleRed (Version 1.02)

Bernard Sufrin

November 24th 2022
(Revised March 26th 2025)

1 AppleRed

1.1 Introduction

Red (also known as **AppleRed**) is a no-frills **UNICODE**-capable *modeless* (and *colourless*) text editor with a simple implementation that can be customized using the **redscript** language, and can be straightforwardly extended using **scala**, and configured using **redscript**¹. It has no pretensions to being an all-encompassing workplace, and unlike many IDE and modern editors does not confuse its user by spontaneously trying to be helpful: everything it does it does in response to user input from mouse, pad, or keyboard.

Its **undo/redo** facility is conventional, but in addition to this it has a *cut ring* that retains the last 240 or so *recently deleted* blocks of contiguous text. This feature is orthogonal to **undo/redo** and makes it straightforward to re-insert material that has – perhaps inadvertently – been deleted from any editing session running in the same editor server. The cut ring interface is similar to an editing session interface, so material can be transferred from it by copy/paste to any application (including **Red**) that has an interface to the system clipboard.

I have used **Red** mainly for preparing **(xe-)latex** and **markdown** manuscripts, as well as for preparing programs that don't warrant the burden of getting to grips with an IDE.

Its support for **latex** manuscripts includes the straightforward manipulation of nested blocks delimited by **\begin{...}** and **\end{...}**.

Its support for **xml** includes the straightforward manipulation of nested blocks delimited by **<id ... >** and **</id>**, as well as **<id ... />** constructs.

¹An extensible dialect of the Lisp family.

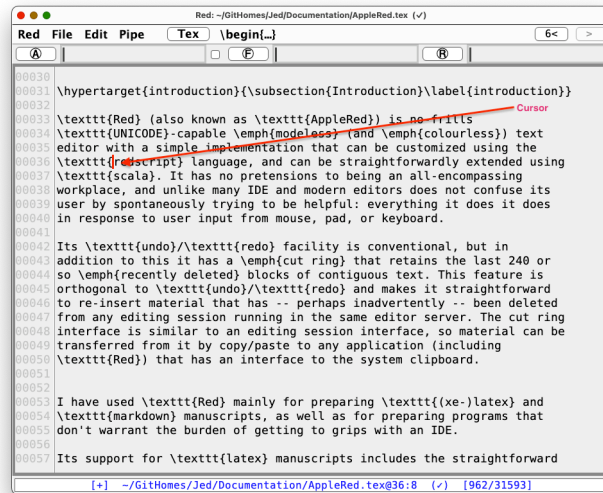


Figure 1: A Session Window

In both cases, when Red/Select is enabled, clicking at the left of the opening of a block, or at the right of the closing of a block will cause that entire block (including nested sub-blocks) to be selected (“tentatively”).

In addition, clicking at the right of any closing bracket will cause all text between that bracket and the corresponding opening bracket to be selected. Dually, clicking at the left of any opening bracket will cause all text between that bracket and the corresponding closing bracket to be selected (“tentatively”). The built-in corresponding pairs are: “{}”, “()”, “[]”, as well as most of the unicode bracketing pairs. Additional pairs can be added straightforwardly.

1.1.1 Tex, Pandoc

The Tex button on the menu bar² generates pdf from the (default tex) source manuscript – usually, but not necessarily, the manuscript being edited – using a script `redpdf` that is expected to be somewhere on the user’s PATH. The `redpdf` I use assumes that the manuscript is in latex format (`xelatex` format if its first line is the comment `%xelatex`), and invokes `pdflatex` (or `xelatex`) with appropriate parameters before invoking the platform pdf viewer (Skim on OS/X, evince on Linux).

²The configuration provided puts this button on the menu bar when a .tex document is being edited.

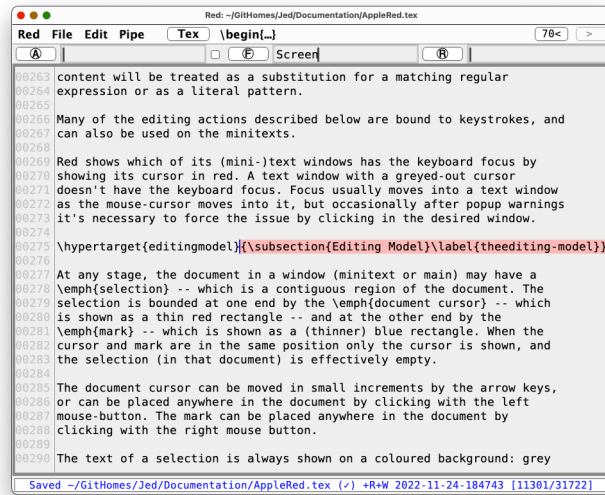


Figure 2: Session Window with tentative selection

A little care has to be taken to ensure that the fonts used in production of the pdf file include glyphs corresponding to all the Unicode characters appearing in the manuscript. The present xelatex manuscript uses the following font setup:

```
\usepackage{amssymb,amsmath}
\usepackage{unicode-math}
\usepackage{longtable,booktabs}
\defaultfontfeatures{Scale=MatchLowercase}
\defaultfontfeatures[\rmfamily]{Ligatures=TeX,Scale=1}
\setmonofont[]{DejaVuSansMono.ttf}
\setmathfont[]{DejaVuSans.ttf}
```

The Pandoc button on the menu bar³ translates the manuscript being edited into .pdf form using the (easily changeable) script `redpandoc`. My personal variant of the script assumes that the manuscript is in the format:

```
markdown+tex_math_dollars+pandoc_title_block
```

It uses `xelatex` as its final `--pdf-engine`; so you can use a wide variety of Unicode symbols directly in the manuscript. For example:

³When a .md document is being edited.

$$e^{i\pi} + 1 = 0$$

yields:

$$e^{i\pi} + 1 = 0$$

It's possible, but pretty pointless, to revert to straight `pdflatex` as the pdf engine: you just deny yourself the use of a fuller range of Unicode characters in your markdown manuscript.

My `redpandoc` script will simultaneously generate an `.html` version of the output if a file `PANDOC-HTML` is present in the source folder; and will generate pdf *via* the html file (rather than directly) if a file `PANDOC-PDF-VIA-HTML` is present.

To set up the same fonts as used in the present xelatex manuscript one would use a title block like this at the head of the markdown manuscript.

```
---
title: AppleRed
fontfamily: DejaVu Sans
mainfont: DejaVuSerif.ttf
sansfont: DejaVuSans.ttf
monofont: DejaVuSansMono.ttf
mathfont: DejaVuSans.ttf
header-includes:
- '\usepackage{unicode-math}\set{=xelatex}'
---
```

1.1.2 SyncTex features

The `redpdf` script invoked by the `Tex` button uses `synctex` features to collaborate with the platform viewer (`Skim` on OS/X, `evince` on Linux).

1. The current editing position in the manuscript being edited is shown as the viewer opens (or reopens) the newly-generated pdf file after the `Tex` button is pressed.
2. The current position in the editing session can be changed from the viewer by the appropriate gesture (From `Skim` this is `Shift-Cmd-click` on one of the pdf lines).

1.2 Installation

The `AppleRed.app` directory can be installed either on OS/X or on Linux by copying it to your local applications folder: `~/Applications`.⁴ Before use it is essential to copy the profile files `.AppleRed*.profile` from

`AppleRed.app/Contents/Resources/Profiles/`

to your `$(HOME)` and, if necessary, adjust them to your own requirements.

The profiles are nearly identical – the important things to change are the PATHs exported from `.AppleRed-application.profile`; the other profiles are (but needn't be) essentially the same. The following command will do the copy:

```
cp -pr $(find ~/Applications/AppleRed.app -name ".AppleRed*.profile") ~
```

At this point it is appropriate to copy the `red` and `cred` scripts, and their friends to a folder somewhere on your shell; or to make symbolic links to them from such a folder. These scripts will be found in the folder:⁵

`AppleRed.app/Contents/Resources/Shellscripts`

1.3 Invocation

1.3.1 Scripts

The `red paths`, and `cred paths` commands work on both operating systems, independently of whether the `AppleRed` app has been started. The former starts the editing server (if it hasn't already been started) which then starts editing sessions for the documents denoted in the filestore by the *paths*. The server runs until the last of its editing sessions is closed; but keeps running in this case if it was started with an empty *paths*.

The latter starts its editing sessions independently of any server. It terminates when its last editing session is closed.

⁴On OS/X you could also copy it to the system applications folder.

⁵A set of scripts is provided that function equally well on Linux and OS/X.

1.3.2 The AppleRed app

On OS/X the app acts as a drop-target, or as an “open-with” subject. When invoked, it starts a editing server (if necessary), then passes its argument to it, then (finally) terminates. Think of it as a “start button” for the server and don’t expect it to hang around between “starts”. Drop the `AppleRed.app` icon onto the OS/X dock if you want to make it easily available.

There are many ways of emulating this sort of functionality on Linux: almost as many as there are Linux distributions. Find a geek to tell you how on yours: I made it happen on my own Ubuntu 22.04.

1.4 Editing Sessions

In each session window you will see the main text editing component in the middle, with three minitext-editing components in a row above it, and a session status component at the bottom. Figure 1 is an example; it also shows the position of the editing cursor.

The minitexts have buttons to their left labelled **(A)** (*Argument*), **(F)** (*Find*) and **(R)** (*Replace*). Pressing one of these buttons clears the adjacent minitext.

The *Argument* minitext provides arguments for many of the other editing actions, including those on the *File* menu which start new editing sessions, allow the document to be saved in a different file, and so forth.

The *Find* and *Replace* minitexts provide arguments for the actions of the same name. The checkbox next to *Find* indicates whether its content will be interpreted as a regular expression or as a literal pattern. The same checkbox indicates whether the *Replace* content will be treated as a substitution for a matching regular expression or as a literal pattern.

Many of the editing actions described below are bound to keystrokes, and can also be used on the minitexts.

Red shows which of its (mini-)text windows has the keyboard focus by showing its cursor in red. A text window with a greyed-out cursor doesn’t have the keyboard focus. Focus usually moves into a text window as the mouse-cursor moves into it, but occasionally after popup warnings it’s necessary to force the issue by clicking in the desired window.

1.5 Editing Model

At any stage, the document in a window (minitext or main) may have a *selection* – which is a contiguous region of the document. The selection is

bounded at one end by the *document cursor* – which is shown as a thin red rectangle – and at the other end by the *mark* – which is shown as a (thinner) blue rectangle. When the cursor and mark are in the same position only the cursor is shown, and the selection (in that document) is effectively empty.

The text of a selection is always shown on a coloured background: grey if the selection was *definite* and pink if the selection was *tentative*. Figure 2 shows a session window with a tentative selection.⁶ See **Treatment of the Selection** for further detail.

The document cursor can be moved in small increments by the arrow keys, or can be placed anywhere in the document by clicking with the left mouse-button. The mark can be placed anywhere in the document by clicking with the right mouse button.

After every editing action the position of the document in the editor window is adjusted automatically so that *the document cursor is visible*. Between editing actions the position of the document in the window can be adjusted manually by the scrollwheel, This can make the document cursor “disappear” temporarily.

“Dragging” the mouse with left button pressed causes a selection to be started (and then extended as the mouse is dragged). If the mouse is dragged outside the window, then the position of the document is adjusted so that it remains visible.

There is a *cut-ring* which holds the last several texts copied (C-C) or deleted from the document. The most recent such text is also placed on the operating system’s clipboard and the most recent material placed in the system clipboard by Red or any other application is accessible and can be *pasted* into the document or *swapped* with the current selection by invoking the the *Paste (Swap)* actions. These are usually bound to the keys C-V, and C-B. The *cut* action deletes the entire selection; it is usually bound to the key C-X. When two ore more adjacent parts of the document are deleted (or cut) in successive actions they are treated as if they had been cut at the same time.

1.6 Editing Commands

A comprehensive set of editing actions are built-in, and bound to a standard set of keys or menu entries by default. Some actions are available on more than one key or menu entry. The actions described below are a selection of those provided.

⁶It is likely that this selection was made when the cursor was moved to the right of the rightmost curly bracket on line 275.

The editor can be customised using the `redscript` language. This supports the definition of new actions as well as changes of binding of actions to keys. The language will be described in a separate section.

Neither AppleRed nor I distinguish between the control-shift and (OS/X) command-shift keys: they appear as `C-` below. This is entirely to preserve my sanity, because I use both Linux and OS/X. Although it reduces the absolute number of keys it is possible to bind to separate actions, there are still enough for any (sane) user – especially one prepared to use the mouse to navigate or hit a menu button occasionally.

1.6.1 Undo and Redo

As editing actions happen their effects are pushed onto the *done stack* for that session.

Undo (`C-z`) undoes the topmost effect on the *done stack* and pushes it onto the *undone stack*.

Redo (`C-Z`) re-does the topmost effect on the *undone stack*; and pops it from that stack.

Redo can also be invoked from the redo button at the right end of the menu bar: it is labelled $n>$, where n is the depth of the *undone stack*. *Undo* can also be invoked from the redo button to the left of the redo button: it is labelled $n<$, where n is the depth of the *done stack*. Consecutive character insertions on the same line (between other actions) are “bunched”, so that their effect can be undone all at once. Consecutive cuts or other deletions of adjacent text are also bunched.

When editing actions other than *Undo* or *Redo* happen, the entire *redo stack* is emptied.⁷ A *Save* writes the document in its current state to the filestore with its current name; and will (usually) preserve one or more backup copies of the file. The undo stack is preserved over a save; but when a save is (notionally) undone nothing happens to the filestore.

1.6.2 Cut and Paste

Cut (`C-x`, or `F1`) removes the selection from the document puts it in the cut-buffer, and adds it to the cut-ring.

Paste (`C-v`, or `F2`) inserts the content of the cut-buffer into the document and re-selects it.

⁷There is in principle no bound on the depth of the *done stack* (I have seen it deeper than 4k. The bunching of insertions and deletions make this feasible; but it is inadvisable to try to *Undo* too far.

Copy (C-c, or F3) replaces the cut-buffer with the selection (if there is a selection).

SwapSel (C-b, or F4) exchanges the selection and the cut-buffer (except that if the selection is null, then the cut-buffer doesn't change). The replaced text is added to the cut ring as if it had been deleted.

SwapCursorAndMark (F5) does what its name suggests.

1.6.3 Find and Replace (Principles)

In what follows:

- *following* means “at a position to the right of or below the *editing cursor*.”
- *preceeding* means “at a position to the left of or above the *editing cursor*.”
- **(F)** means “the pattern specified by the *Find* minitext.”
- **(R)** means “the replacement specified by the *Replace* minitext.”

If the checkbox adjacent to the *Find* minitext is checked then the find text is interpreted as a (Java-style) regular expression. Otherwise it is interpreted literally.

When the find text is interpreted as a regular expression, the replacement text is interpreted as the text obtained (using standard Java regular-expression substitution rules) by substituting each instance of $\$n$ in it by the text that matches the n th bracketed expression in the instance of the pattern that is being replaced. Otherwise it is interpreted literally.

- *FindDown* (F7, or Numpad-0) selects the nearest *following* instance of **(F)**
- *FindUp* (Shift-F7), or Shift-Numpad-0) selects the nearest *preceeding* instance of **(F)**
- *FindSelDown* (Alt-f7, or Alt-Numpad-0) makes the current selection the *Find* pattern, turns off regular expression interpretation, and then acts as *FindDown*.
- *FindSelUp* (Shift-Alt-F7, or Shift-Alt-Numpad-0) makes the current selection the *Find* pattern, then acts as *FindUp*.

- *ClearFind* (C-Numpad-0, or Button(F)) clears the find minitext, and moves the editing cursor to that text.
- *ClearRepl* (C-Numpad-., or Button(R)) clears the replace minitext, and moves the editing cursor to that text.
- *ClearArg* (C-Numpad-Enter, or Button(A)) clears the argument minitext, and moves the editing cursor to that text.

If the current selection is an instance of **(F)** then:

- *ReplaceDown* (C-r), Numpad-. saves the current selection in the cut-buffer and replaces it with the **(R)** selecting the replacement text and leaving the cursor to the right of the mark.
- *ReplaceUp* (C-R), Shift-Numpad-. saves the current selection in the cut-buffer and replaces it with the **(R)** selecting the replacement text and leaving the cursor to the left of the mark.

The C-f key behaves as much as possible like its behaviour in more conventional applications. Its behaviour is summarized by the following table, in which the selection is denoted **(S)**, and **FIND** means selecting the nearest **(F)** in the direction indicated by the shift key (upwards if it is pressed).⁸

(S)	(F)		Effect
nonempty	empty	with Alt shift otherwise	FIND
nonempty	nonempty		(F):= (S); FIND
nonempty	nonempty		FIND
empty	nonempty		(F):= (S); FIND
empty	empty		move editing cursor to (F)

1.6.4 Find and Replace (Technique)

Many editors provide a multiplicity of modes for finding and replacing, and some include the possibility of “approving” replacements. In what follows we explain our simpler approach.

A way of replacing the next instance of "F00" with "BAR", starting from scratch, without using the mouse to move between texts is to type:

C-Numpad-0 F 0 0 Numpad-0 C-Numpad-. B A R Numpad-.

⁸The effect of C-f while editing **(F)** is also **FIND**.

Because the *Find* and *Repl* minitexts are now set to the right pattern and replacement, replacing the following instance just requires the two keystrokes *FindDown ReplaceDown*, that is, either:

`Numpad-0 Numpad-.` or `F7 F8` or `C-f C-r`

Subsequent instances can be found/replaced with the same keystrokes; and the same technique can be used “upwards”, by using *FindUp* and *ReplaceUp* (the same keys pressed, but shifted).

- *ReplaceAll* ((F)->(R) on the edit menu) replaces (without any interaction) all instances of the *Find* minitext in the current selection with the *Repl* minitext, and selects the resulting transformed text.

The original selection is preserved in the cut buffer (and therefore in the cut ring), so this action can be undone immediately with *SwapSel* if you are immediately overcome with remorse afterwards, and by browsing in the cut ring if the remorse takes longer to arrive.

If you want to “approve” each replacement interactively, then just use *FindDown ReplaceDown* in sequence repeatedly, undoing any replacement you don’t approve of with *SwapSel* (or *Undo*).

To find/replace earlier instances just type the same actions with the shift key pressed.⁹

1.6.5 Indentation and the Tab key

When there is no selection **Tab** inserts enough spaces to move the cursor so that it is a multiple of 8 characters from the left margin.

When **Red/Autoindent** is enabled, it is assumed that the indentation of the text following a newline is to be the same as the indentation of the current line. This is achieved by inserting spaces after **Enter** is typed, and removing spaces to the right of the cursor if necessary.

When there is a *nonempty selection*:

- *Indent selection* (**Tab**, **C-Right**) indents the selected lines by 1 space. If invoked with the **alt**-shift pressed, then the selected lines are indented by prefixing them with the (**A**) text, and then re-selected.

⁹You may have noticed that the “upward” variant of an action is bound to the shifted key that its “downward” variant is bound to.

- *Undent selection* (**Shift-Tab**, **C-Left**) (when there is a nonempty selection) reduces the indentation of the selected lines by 1 space where possible. If invoked with the **alt-shift** pressed, then the selected lines are “undented” by the (literal) **(A)** text where possible, and then re-selected.

The indent and undent selection actions effectively *cut* the selection and replace it with the indented (undented) text. A consecutive sequence of *Indent* or *Undent* actions is undone by *Undo*.

1.7 Treatment of the Selection

Red offers two modes of treating the selection. Most users will decide on one of them and stick to it forever. In what follows, a “tentative” selection means one made by auto-selecting material between matching brackets.

Having grown accustomed to the second mode in my homegrown editors for more than thirty years I was reluctant to adopt the first: but after implementing the differential treatment of “tentative” and “definite” selections I changed my mind.

1. **Typing-cuts-selection:** Typing new material automatically cuts a “definite” selection into the cut buffer (and adds it to the cut ring). This is (almost) the behaviour that most people have come to expect of editors; except that it does not cut “tentative” selections.
2. **Typing-removes-mark:** Typing new characters removes the mark, and thus deselects the *selection*, but does not delete the selected material. In this mode Red does not distinguish between “tentative” and “definite” selections.

The choice of modes is made using the **Red/Typeover** checkbox. As with most other preferences its value is preserved between **Red** invocations.

Tentative selections are made when a closing bracket (of some kind) is typed, or when the mouse is clicked just to the left of an opening bracket or to the right of a closing bracket. In both cases the scope of the selection is the well-nested bracketed text (if there is one) between the opening and closing bracket. By well-nested we mean well-nested with respect to the opening (closing) bracket of a bracketing pair.

Red builds-in most unicode bracketing pairs in addition to the obvious “{}”, “()”, “[]” and can be easily adapted to add more pairs specified by regular

expressions; for example those that match the built-in Latex and HTML block brackets described below.¹⁰

Any `\begin{kindb}` is considered to match any `\end{kinde}` no matter whether *kind_b* and *kind_e* are the same. Likewise any `<kindb ...>` and `<kinde>`.

Tentative selection behaviours are enabled or disabled by the `Red/Select/(...@)` and `Red/Select/{...}` menu entries respectively: their values are indicated on the menu entries and preserved between `Red` invocations.

1.8 Abbreviations

Red can be configured with *abbreviations* for texts that contain unicode symbols that may not appear directly on the keyboard.

One form of abbreviation is built-in, namely `\uxxxx`, where each of the four *x* is a hex digit. It abbreviates the Unicode glyph with that code.

The action *Abbrev* (ESC) finds the longest abbreviation that appears just before the cursor, and replaces it with the text it abbreviates.

The action *Unabbrev* (Alt-Shift-ESC) replaces the character immediately to the left of the cursor with its unicode encoding, in the form `\uxxxx`.

1.8.1 Examples

Here are extracts from the (extensive) bindings file `symbols.redscript` delivered with Red. Notice that both unicode glyphs and their numbered Unicode representations in the form `\uXXXX` can be used in abbreviations and their substitutions.

<code>abbrev "lnot"</code>	<code>"\u00AC"</code>	<code># ¬</code>
<code>abbrev "/\u2203"</code>	<code>"∄"</code>	
<code>abbrev "land"</code>	<code>"\u2227"</code>	<code># ∧</code>
<code>abbrev "/\"</code>	<code>"\u2227"</code>	
<code>abbrev "=>"</code>	<code>"⇒"</code>	
<code>abbrev "<=>"</code>	<code>"⇔"</code>	
<code>abbrev "lor"</code>	<code>"∨"</code>	<code># ∨</code>
<code>abbrev "\\/"</code>	<code>"\u2228"</code>	
<code>abbrev "cap"</code>	<code>"\u2229"</code>	
<code>abbrev "cup"</code>	<code>"\u222a"</code>	
<code>abbrev "euro"</code>	<code>"€"</code>	
<code>abbrev "pound"</code>	<code>"\u20a4"</code>	

¹⁰The adaptation may require thought to be given to the potential for ambiguity, and for the ambiguity to be resolved programmatically.

Words and phrases can also be abbreviated

abbrev "lb"	"pound"
abbrev "lbs"	"pounds and pounds"

Further details of binding configurations will (eventually) be found in an appendix.

A APPENDIX: Bindings and Abbreviations

A.1 RedScript

As the editor is started it reads the file `~/.red/bindings.redscript`: the role of the script in this file is to provide information needed by each editing session. There is a good deal of room for customization here, but the distributed version of the script provides a decent minimum collection of features that permit some choices to be made by the user.

B APPENDIX: History of AppleRed

Red starts life as a toy: **Red** started life as a program to use as an exemplar for teaching first-year Oxford undergraduates a short course in object-oriented programming using `scala`. It was to be the basis of a number of practical exercises.

Its editing model was identical to that used in the **dred** editor I had built from scratch in `java` in the early 2000's and had been using ever since. **Dred** had developed from a sequence of editors, all using the same *modeless* editing model derived from an abstract formal specification I had published in the late 1970's.

Red becomes my editor of choice: It wasn't long before I decided that it would be a good idea to make **Red** more than a toy; and soon after I started its development (during the first pandemic lockdown period) it became my editor of choice. Shortly afterwards I found it straightforward to make it integrable with `OS/X` as an app, as well as runnable as an ordinary JVM-compatible program on `OS/X`, `Linux`, and `Windows`.

My intention was to remove some of its more advanced features for use in the course, and this is what I did: the result was a simpler editor, **jed**, published for my students as the course started in Trinity term in the spring of 2022. For reasons of pedagogy that editor incorporated `undo/redo`.

Red inherits undo/redo from Jed: The `undo/redo` feature of **jed** had not been implemented in the original **Red**, because the latter's cut-ring feature met all my reasonable needs. But as soon as the course was over (early summer 2022) I started to enhance the original **Red** with `undo/redo`.

Apple changes its security model: The next spasm of **Red**'s development was occasioned by a change in the `OS/X` security model for apps. To

paraphrase my obituary for the unitary **AppleRed** app that provided editing sessions (on OS/X and Linux) through an embedded server as well as providing the more usual *drag-drop*, and *open-with* functionality:

“On **Catalina** (versions above 10.14) and its successors the situation is complicated by (welcome, but ferocious) Apple security measures that *forbid* the reception of messages (of any kind) by uncertified apps. At the time of writing I see no reasonable way round these measures (and, believe me, I have looked) that can be taken without paying the **Apple Ransom**, which imposes both a financial and an intellectual burden. I have therefore decided to distribute the program as a simple OS/X app that invokes a helper program to provide editing session functionality.”

But by dint of the simple but legitimate technique (outlined in the obituary, and explained in the source code) I nevertheless managed to deliver all functionality of an OS/X app on OS/X.

Note that there had never been an analogous problem caused by security measures on Linux.

Customization gives rise to RedScript: A much more recent development was a dramatic increase in the sophistication of the notation used for customising the editor. **RedScript** is a straightforward language, reminiscent of **scheme**, that “collaborates” with the editor code to provide the editing context: the content of menus, and the bindings of keys to actions. Its facilities are still evolving as its potential is explored.